THE DISCRETE FOURIER TRANSFORM OVER PRIME FIELDS BELOW $N \log N$ ON MULTITAPE MACHINES

DAN BROWN* AND KENNY DANIEL

Abstract. A fixed finite multi-stack Turing machine, in the model `Turing.FinTM2` of Lean's Mathlib, computes the discrete Fourier transform of every length n over every prime field $\mathbb{F}_p$, given p, a primitive n-th root of unity and the n entries in binary of fixed width, in worst-case time $O\big(N(\log N)^{1-\kappa}\big)$, where N is the input length and $\kappa = 111364923 \cdot 2^{-46} \approx 2^{-19.27}$. The saving κ is inherited from a formally verified integer multiplier on the same machine model. The reduction is the standard reduction of the transform to one integer product: Bluestein's chirp, written with binomial coefficients and the inverse root ω^{n-1}, Kronecker packing, and Barrett reduction with a reciprocal computed by Newton's iteration. The theorem is machine-verified in Lean 4 over Mathlib. Its statement uses Mathlib definitions only, and its proof depends only on Lean's three standard axioms.

1. Introduction

For a prime p, an integer $n \geq 1$, a primitive n-th root of unity $\omega \in \mathbb{F}_p$ and a vector $a \in \mathbb{F}_p^n$, the discrete Fourier transform is

$$\mathrm{DFT}_\omega(a)_k = \sum_{j=0}^{n-1} a_j\, \omega^{jk} \qquad (0 \leq k < n).$$

When n is smooth, the fast Fourier transform computes it with $O(n \log n)$ field operations. The bit complexity on a multitape Turing machine is a different question: the field elements have $w = \mathrm{size}(p)$ bits, n need not be smooth, and the machine moves its data along tapes. The classical route reduces the transform to a single integer multiplication of $O(N)$ bits, where $N \asymp nw$ is the input length. The transform then costs the time of that product, plus $O(N)$ for the surrounding passes and $O(n)$ multiplications of $O(w)$-bit numbers.

Integer multiplication on multitape machines takes time $O(n \log n)$ by Harvey and van der Hoeven [3], and time $O\big(n(\log n)^{1-\kappa}\big)$ for a positive κ by the OpenAI manuscript [7]. Kenny Daniel's repository `integer-mult-bounds-lean` [4] formalizes the bound of [7] in Lean, for one fixed machine in Mathlib's multi-stack model. Its theorem `MathlibTM2.mulInTime` gives $\kappa = 111364923 \cdot 2^{-46}$, the layout margin $\varepsilon c(1-\tau)$ rounded down to a multiple of 2^{-46}, with $\varepsilon = 31/64$, $c = 7/8$ and $1 - \tau = 3734039 \cdot 10^{-12}$, where τ is the exponent of its bit interchange, the two-stage partial-swap word of PR97. Its complex transforms come from the T16 node, whose branching exponent is below $999/1000$. The reduction carries the same saving to the transform.

Theorem. *There is a fixed finite multi-stack machine that computes* $\mathrm{DFT}_\omega(a)$ *for every prime* p*, every* n*, every primitive* n*-th root of unity* $\omega \in \mathbb{F}_p$ *and every* $a \in \mathbb{F}_p^n$*, with the input written as* p*,* ω*,* $a_0, \ldots, a_{n-1}$ *in exactly* $\mathrm{size}(p)$ *binary digits each, in time*

$$O\big(N(\log N)^{1-\kappa}\big), \qquad \kappa = 111364923 \cdot 2^{-46},$$

where $N = (n+2)(\mathrm{size}(p)+1)$ *is the input length. The implicit constant depends neither on* p *nor on* n*. More generally, every multiplier of this kind with saving* $\kappa \leq 1$ *gives a transform machine with the same saving.*

Date: October 18, 2026.

*Author of *The Da Vinci Code*.

The reduction is the standard reduction of the discrete Fourier transform to integer multiplication. The contributions of this paper are the explicit bound on multitape machines, inherited from the Lean multiplication theorem, and a machine-checked statement of it over Mathlib definitions only. The bound is uniform in the relation between n and p: it covers $n = 2$ with p of millions of bits as well as $n = p - 1$. The first regime requires every one-time setup on p to cost $O(M(w))$, the time of one product of w-bit numbers, and this decides the choice of the modular reduction (Section 4.5).

Section 2 gives the statement in Lean, and Section 3 explains why the inputs have fixed width. Section 4 gives the reduction, Section 5 the machine and its cost accounting, and Section 6 the formalization.

2. The statement

The Lean statement is `DFTStatement` in `MathlibTM2/DFT/Statement.lean`, a file that imports Mathlib only. Apart from the few definitions shown, everything in it is Mathlib vocabulary: the alphabet Γ', the encoding `encodeNat` of $\mathbb{N}$ in binary, `ZMod`, `IsPrimitiveRoot`, `TM2ComputableInTime` and the asymptotic relation `=O[atTop]`. Numbers are written least significant bit first, each followed by a comma.

```
def encodeFixed (w x : ℕ) : List Γ' :=
  List.ofFn fun i : Fin w => Γ'.bit (x.testBit i)

def encodeFixedList (w : ℕ) (l : List ℕ) : List Γ' :=
  l.flatMap fun x => encodeFixed w x ++ [Γ'.comma]

def encodeNatList (l : List ℕ) : List Γ' :=
  l.flatMap fun x => (encodeNat x).map inclusionBoolΓ' ++ [Γ'.comma]

def dft {p n : ℕ} (ω : ZMod p) (a : Fin n → ZMod p) (k : Fin n) : ZMod p :=
  ∑ j, a j * ω ^ ((j : ℕ) * k)

structure DFTInstance where
  p : ℕ
  [prime : Fact p.Prime]
  n : ℕ
  ω : ZMod p
  primitive : IsPrimitiveRoot ω n
  a : Fin n → ZMod p

def DFTInstance.encode (x : DFTInstance) : List Γ' :=
  encodeFixedList (Nat.size x.p) (x.p :: x.ω.val :: List.ofFn fun j => (x.a j).val)

def DFTInstance.transform (x : DFTInstance) : List ℕ :=
  List.ofFn fun k => (dft x.ω x.a k).val

def DFTInTime (κ : ℝ) : Prop :=
  ∃ M : TM2ComputableInTime DFTInstance.encode encodeNatList DFTInstance.transform,
    (fun N : ℕ => (M.time N : ℝ)) =O[atTop]
      fun N : ℕ => (N : ℝ) * Real.log N ^ (1 - κ)

def DFTStatement : Prop := ∃ κ : ℝ, 0 < κ ∧ DFTInTime κ
```

`TM2ComputableInTime` is Mathlib's notion: a machine of type `Turing.FinTM2`, a time function of the input length, and a proof that on every input the machine halts with the encoded output within that

time. The time function therefore bounds the worst case over all instances of a given length. The output is the list of canonical representatives $\mathrm{DFT}_\omega(a)_k \in \{0, \dots, p-1\}$ in shortest binary form; its length is at most $n(\mathrm{size}(p)+1) \le N$. The file checks `encodeFixedList`, `encodeNatList` and one transform over $\mathbb{F}_5$ by `decide`.

3. Why the inputs have fixed width

With shortest binary forms on input as well, no bound $O\big(N(\log N)^{1-\kappa}\big)$ is possible. Take $n = p-1$, a generator ω of $\mathbb{F}_p^\times$, and $a = (0, 1, 0, \dots, 0)$. In shortest form each zero entry is a single comma, so the input has about $p + 2\log_2 p$ symbols. The transform is $\mathrm{DFT}_\omega(a)_k = \omega^k$, which runs over all of $\mathbb{F}_p^\times$, so the output has $\sum_{x=1}^{p-1}(\mathrm{size}(x)+1) \asymp p \log p$ symbols. A machine writes at most one output symbol per step, so its time on these inputs is at least of order $N \log N$.

With every entry written in exactly $\mathrm{size}(p)$ digits, the output is never longer than the input, and the lower bound disappears. Fixed width is also the usual format for vectors over $\mathbb{F}_p$.

4. The reduction

Fix an instance and write $w = \mathrm{size}(p)$, so $2^{w-1} \le p < 2^w$. For $n \ge 1$, ω has order n in $\mathbb{F}_p^\times$, so $n \mid p-1$ and $n < p$ (`DFTInstance.n_lt_p`). No case of n needs separate treatment: for $n = 0$ every step below runs zero rounds and the output is empty.

4.1. **The chirp without a square root.** Bluestein [2] turns the transform into a correlation through the identity $jk = \frac{1}{2}\big((j+k)^2 - j^2 - k^2\big)$, which needs a square root of ω. The binomial form avoids it:

$$jk = \binom{j+k}{2} - \binom{j}{2} - \binom{k}{2},$$

an identity of integers (`choose_two_add`). Put $\omega' = \omega^{n-1}$, so $\omega'\omega = 1$, and $c_m = \omega^{\binom{m}{2}}$, $c'_m = \omega'^{\binom{m}{2}}$. Then $\omega^{jk} = c'_k\, c'_j\, c_{j+k}$ (`chirp_identity`), and

$$\mathrm{DFT}_\omega(a)_k = c'_k \sum_{j=0}^{n-1} u_j\, c_{j+k}, \qquad u_j = a_j\, c'_j.$$

The sum is a correlation of the chirped input u, of length n, with the chirp c. Both chirps follow from $\binom{m+1}{2} = \binom{m}{2} + m$: $c_{m+1} = c_m\, \omega^m$ and $\omega^{m+1} = \omega^m \omega$, one product modulo p each, and the same with ω'.

4.2. **Kronecker packing.** Let $\bar u_j, \bar c_m \in \{0, \dots, p-1\}$ be the representatives and $B = 2^s$ with $s = 3w$. Since $n < p \le 2^w$, every sum of n products of representatives is below $n(p-1)^2 < 2^{3w} = B$. Pack the reversed chirped input and $2n$ values of the chirp:

$$U = \sum_{i=0}^{n-1} \bar u_{n-1-i}\, B^i, \qquad C = \sum_{m=0}^{2n-1} \bar c_m\, B^m.$$

The coefficient of B^{n-1+k} in the polynomial product, for $k < n$, is the correlation $T_k = \sum_{j=0}^{n-1} \bar u_j\, \bar c_{j+k}$, and every coefficient of the product is a sum of at most n terms below $(p-1)^2$, hence below B. So no carries occur, and T_k is a digit of the integer BUC in base B:

$$T_k = \big\lfloor BUC / B^{n+k} \big\rfloor \bmod B.$$

The extra factor B places T_0 at an index the machine reaches by dropping $|U| = ns$ bits. The two factors have $(n+1)s$ and $2ns$ bits, so the single product has input length at most $(3n+1)s + O(1) = O(N)$. Finally $\mathrm{DFT}_\omega(a)_k = c'_k\, T_k$ in $\mathbb{F}_p$. `fastDFT_transform` states this computation on natural numbers, with all reductions modulo p, and identifies its output with `DFTInstance.transform`.

4.3. **The Newton reciprocal.** Every reduction modulo p is a Barrett reduction [1], which needs the reciprocal $\mu = \lfloor 2^{s'}/p \rfloor$ for one exponent $s' > 3w$. It is computed by Newton's iteration with precision doubling [5, §4.3.3], on the normalized divisor $D = 2^{4w}p$ of $5w$ bits. For $2^{N-1} \le D < 2^N$ let

$$\mu_k = \left\lfloor 2^{2k} / \lfloor D/2^{N-k} \rfloor \right\rfloor,$$

the reciprocal of the top k bits of D, so $\mu_1 = 4$. From $a = \mu_k$ and the top $2k$ bits d of D, the guess

$$y = 2^{k+1}a - \left(\lfloor da^2/4^k \rfloor + 1\right)$$

satisfies $yd < 16^k < (y+9)d$ (`newton_step`), so μ_{2k} is y plus at most eight corrections (`newton_level_arith`). One level costs three products of $O(k)$-bit numbers and $O(k)$ further steps. The levels run while a unary counter of $r = 2w$ symbols, which loses k symbols per level as k doubles, is nonempty. The last precision k' then lies in $(2w, 4w]$, the sum of the precisions over all levels is at most $2r + 1$, and the whole iteration costs $O(M(w))$. For $k' \ge w$ the top k' bits of D are $2^{k'-w}p$, so $\mu_{k'} = \lfloor 2^{k'+w}/p \rfloor$ (`recip_value`), the reciprocal for $s' = k' + w > 3w$.

4.4. **Barrett reduction.** With $\mu = \lfloor 2^{s'}/p \rfloor$ and $x < 2^{s'}$, the quotient $q = \lfloor x\mu/2^{s'} \rfloor$ satisfies

$$qp \le x < qp + 2p$$

(`barrett2_le`, `barrett2_lt`), so x mod p is $x - qp$ or $x - qp - p$. One reduction costs two products of $O(w)$-bit numbers and $O(w)$ further steps. Every value the pipeline reduces is below $2^{3w+1} \le 2^{s'}$: products of two residues are below 2^{2w}, and the correlations T_k are below $B = 2^{3w}$.

4.5. **Montgomery reduction in the tiny-n regime.** Montgomery's reduction [6] replaces division by p with division by $R = 2^r$, given the constant $-p^{-1}$ mod R. That constant costs $O(M(w))$ by Hensel lifting $y \mapsto y(2 + py)$, again a geometric sum. Entering Montgomery form, however, needs R^2 mod p. From R mod p it takes either r modular doublings, $O(r)$ bit operations each and $O(r^2)$ in all, or a Montgomery power with $\log_2 r$ squarings of cost $M(r)$ each, $O(M(w) \log w)$ for $r = O(w)$. The budget of the transform is $O\big(N(\log N)^{1-\kappa}\big)$ with $N = \Theta(nw)$, about $n\,M(w)$, and both costs exceed it when $n < \log w$. Computing R^2 mod p within $O(M(w))$ amounts to a division by p, and once the reciprocal μ is available, Barrett reduction performs every reduction of the pipeline directly. The formalization proves a Montgomery multiplication and the Hensel inverse as well; the final theorem uses neither.

5. The machine and its cost

5.1. **The machine.** All code is written in the structured stack language of [4]: programs built from stack actions, sequencing, conditionals, loops and procedure calls, with an execution relation that records exact step counts and compiles to a `Turing.FinTM2` machine. The host register file of [4] provides registers holding binary numbers, least significant bit first, with verified procedures for copying, comparison, addition, subtraction, widening and truncation.

The transform machine adds two stacks over Γ', for its input and its output, around a host register file of $T + 50$ registers: the T registers of the multiplier, followed by 50 registers of the transform. Its procedure table holds the main program and, embedded on the first T registers, every procedure of the multiplier. `mulCall_exec` is the multiplier as a subroutine: from x and y in two registers of widths L_a and L_b, it puts xy into a third register in

$$30(L_a + L_b) + 4\ \mathrm{size}(xy) + m + 63$$

steps, where $m + 2 \le M.\mathrm{time}(\mathrm{size}\,x + \mathrm{size}\,y)$ is the multiplier's own time; every other stack is kept. Here M is a `HostMultiplier` κ, a structure holding the multiplier's procedure table, its time function, the contract that it multiplies every pair within that time, and the bound $M.\mathrm{time}(n) = O\big(n(\log n)^{1-\kappa}\big)$. Nothing in the construction depends on the value of κ.

5.2. **The pipeline.** The main program `dftMain` reads the input onto three registers (p, ω and the entries), runs the pipeline, clears its work registers, prints the output register and restores its initial local state. The pipeline has four loops, each over a counter register, sharing one loop lemma (`countLoop_exec`).

(1) **Setup.** Width registers of $2w$ and $3w$ symbols, and the Newton reciprocal (`recip_exec`): $\mu = \lfloor 2^{k'+w}/p \rfloor$ with $2w < k' \le 4w$, in at most $(7200 + 27H(4r+4))(2r+1) + 20r + 10w + 40$ steps for $r = 2w$, where H is a monotone function with $M.\mathrm{time}(m) \le (m+1)H(m)$.
(2) **Chirps.** Loop A runs n rounds over a copy of the entries. Round m saves ω^m as the candidate inverse root, pushes c_m widened to s bits onto the chirp pack, and updates c and ω^m by two Barrett multiplications (`modMul_exec`). After it the saved value is $\omega^{n-1} = \omega'$. Loop B runs n more rounds, after which the pack holds C.
(3) **Chirped input.** The input loop takes the entries in order, multiplies a_j by c'_j, pushes the result widened to s bits onto U, and updates c' and ω'^j. Pushing on top reverses the order, and the register ends with U exactly. Each round also adds a tick to a counter of n.
(4) **The product.** s zero bits pushed onto the least significant end of U give BU, one call to the multiplier gives BUC, and dropping ns bits leaves the digits from index n on.
(5) **Unpacking.** The last loop takes the low s bits of the remaining product as T_k, reduces it modulo p by Barrett's method, multiplies by c'_k, and emits the result without high zeros onto the output register.

Each round costs $O(k')$ steps plus a bounded number of products of $O(w)$-bit numbers. The round bounds are explicit: with $h = H(3k+3)$, a chirp round costs at most $(1130 + 18h)k + 1500 + 24h$ steps, and a Barrett multiplication at most $(520 + 9h)k + 700 + 12h$.

5.3. **The step bound.** `dftMain_exec` proves that on every instance x, the main program runs from x's encoding alone on the input stack to `encodeNatList x.transform` alone on the output stack in at most $\mathrm{dftBound}(H, n, w)$ steps. The bound is the sum of the parsing cost $4N + 8w + 14$, the pipeline bound, the cleanup cost $100w(n+1) + 100$, and the printing cost $2n(w+1) + 3$. Every argument of H in it is at most $16N + 16$, so

$$\mathrm{dftBound}(H, n, w) + 2 \;\le\; \mathrm{dftTimeOf}(H, N) = 100000\,(N+1)\big(H(16N+16)+1\big)$$

at $N = (n+2)(w+1)$ (`dftBound_le`). The majorant H is built from the multiplier's time function:

$$t_H(m) = \max_{m' \le m} \left\lceil \frac{M.\mathrm{time}(m')}{m'+1} \right\rceil,$$

which is monotone and satisfies $M.\mathrm{time}(m) \le (m+1)\,t_H(m)$. From $M.\mathrm{time}(m) = O\big(m(\log m)^{1-\kappa}\big)$, $t_H(m) \le K + c(\log m)^{1-\kappa} + 1$ for constants K and c (`tH_le`), and so $\mathrm{dftTimeOf}(t_H, N) = O\big(N(\log N)^{1-\kappa}\big)$ (`dftTime_isBigO`). This last step requires $\kappa \le 1$: the factor $t_H + 1$ contributes terms of order N, and these are absorbed into $N(\log N)^{1-\kappa}$ only when $(\log N)^{1-\kappa} \ge 1$. The condition costs nothing. A fixed machine writes a bounded number of output symbols per step, and some inputs of every length n have products of length of order n, so the time of a multiplier is at least of order n and its saving is at most 1.

6. Formalization

The proof is in `IntegerMultBounds/MathlibTM2/DFT/` of the repository [4]: 42 modules and about 9,100 lines. The layers are as follows.

- `Statement`, `Algorithm`: the statement of Section 2 and the arithmetic of Section 4 on natural numbers. `chirp_identity` is the binomial form of Bluestein's identity with $\omega'\omega = 1$, and `fastDFT_transform` identifies the computation with `DFTInstance.transform` when the base B exceeds $n(p-1)^2$.

- `Keys`, `IO`, `Fields`, `Parse`: the machine with two Γ' stacks around the host register file, the transfer of host programs, reading the fixed-width input and printing the output, and the packaging into `TM2ComputableInTime` and `DFTInTime`.
- `MulIO`, `MulCall`: `HostMultiplier` and the multiplier as a subroutine (`mulCall_exec`).
- `Newton`, `NewtonHost`, `NewtonSetup`, `NewtonBlocks`, `NewtonGuess`, `NewtonLevel` and `NewtonLoop`: the Newton reciprocal (`recip_exec`).
- `Barrett`: Barrett reduction and multiplication modulo p (`barrett_exec`, `modMul_exec`).
- `CountLoop`, `PipeMath`, `PipeHost`, `PipeSetup`, `Regs`, `Chirp`, `ChirpLoop`, `Input`, `Unpack`, `PipeA`, `PipeB`, `PipeC`, `Pipeline`, `Output`, `Main`: the pipeline of Section 5 and `dftMain_exec`.
- `Top`: the majorant t_H, the time function, `ComputesWithin`, the asymptotic bound and `HostMultiplier.dftInTime`.
- `DFTCheck`: the final multiplier as a `HostMultiplier` and the final theorem.

The register proofs follow one pattern: each block of host code has a lemma giving its exact effect on the registers it touches and a bound on its steps, and the compositions above it evaluate these effects. The 50 registers of the pipeline are numerals of `Fin 50`, so the disequalities between them are decided by simplification. The top of the development consists of the following theorems, the first two with their proofs omitted:

```
theorem HostMultiplier.dftInTime (M : HostMultiplier κ) (hκ : κ ≤ 1) :
    DFTInTime κ

theorem finalHostMultiplier : Nonempty (HostMultiplier (111364923 / 2 ^ 46 : ℝ))

theorem dftInTime : DFTInTime (111364923 / 2 ^ 46 : ℝ) := by
  obtain ⟨M⟩ := finalHostMultiplier
  exact M.dftInTime (by norm_num)

theorem dft_in_time : DFTStatement :=
  ⟨111364923 / 2 ^ 46, by norm_num, dftInTime⟩
```

`dftInTime` carries the explicit saving, and `dft_in_time` is the existential statement of Section 2, in the same way as `MathlibTM2.mulInTime` and `MathlibTM2.statement` for the multiplier.

`finalHostMultiplier` assembles the multiplier of `MathlibTM2.mulInTime` from public lemmas of [4]: the execution of its host table on all sufficiently long inputs, its time bound, and a dispatcher that extends it to every input. The dispatcher reads up to n_0 input symbols into its finite local state; on a shorter input it writes the product from that state, and otherwise it restores the input and calls the multiplier. `#print axioms` reports `propext`, `Classical.choice` and `Quot.sound`, and nothing else, for both `dftInTime` and `dft_in_time`. The statement file imports Mathlib only, so an auditor of the result reads `Statement.lean` and this axiom report.

7. Remarks

Remark 7.1 (Tiny n, huge p)**.** *For $n = 2$ the input length is $N = 4(w+1)$, and the budget $O\left(w(\log w)^{1-\kappa}\right)$ is the time of a constant number of products of w-bit numbers. Every step of the pipeline fits: the Newton reciprocal is a geometric sum of products, and the loops use $O(n)$ products. A setup with $\Theta(\log w)$ products of w-bit numbers does not fit, which is the reason for Section 4.5. For n close to p the single product BUC dominates.*

Remark 7.2 (Constants)**.** *The constants of `dftTimeOf` are generous: the factor* 100000 *and the argument* $16N + 16$ *cover the sum of all round bounds with room to spare. The leading behaviour comes from the*

multiplier, through $t_H(16N + 16) = O\big((\log N)^{1-\kappa}\big)$*, and the transform adds a constant factor to it. No effort was made to tighten that factor.*

Remark 7.3 (Other savings)**.** *The construction takes the multiplier as a parameter. Any formally verified multiplier in the same model with saving* $\kappa \le 1$ *yields the transform with the same saving through* `HostMultiplier.dftInTime`*, and an improvement of* κ *in* [4] *passes to the transform by changing* `DFTCheck.lean` *alone.*

Remark 7.4 (What is not proved)**.** *The theorem gives an upper bound. It says nothing about lower bounds for the transform, and its input format is the fixed-width format of Section 3. The machine handles every length n for which a primitive n-th root of unity exists in* $\mathbb{F}_p$*, smooth or not. Composite moduli and extension fields are outside its scope.*

References

[1] P. Barrett, *Implementing the Rivest Shamir and Adleman public key encryption algorithm on a standard digital signal processor*, Advances in Cryptology – CRYPTO '86, Lecture Notes in Comput. Sci. **263**, Springer, 1987, 311–323. doi:10.1007/3-540-47721-7_24.

[2] L. I. Bluestein, *A linear filtering approach to the computation of discrete Fourier transform*, IEEE Trans. Audio Electroacoust. **18** (1970), no. 4, 451–455. doi:10.1109/TAU.1970.1162132.

[3] D. Harvey, J. van der Hoeven, *Integer multiplication in time* $O(n \log n)$, Ann. of Math. (2) **193** (2021), no. 2, 563–617. doi:10.4007/annals.2021.193.2.4.

[4] K. Daniel, `integer-mult-bounds-lean`: integer multiplication below $n \log n$ in Lean on Mathlib's finite Turing machines, https://github.com/platypii/integer-mult-bounds-lean.

[5] D. E. Knuth, *The Art of Computer Programming*, Vol. 2: *Seminumerical Algorithms*, 3rd ed., Addison-Wesley, 1997, ISBN 0-201-89684-2. https://www-cs-faculty.stanford.edu/~knuth/taocp.html.

[6] P. L. Montgomery, *Modular multiplication without trial division*, Math. Comp. **44** (1985), no. 170, 519–521. doi:10.1090/S0025-5718-1985-0777282-X.

[7] OpenAI, *Integer multiplication below* $n \log n$, OpenAI Math Release preprint, September 23, 2026, https://github.com/openai/math/blob/main/preprints/Integer-multiplication-below-n-log-n-September-23-2026/paper.pdf.

Email address: jdb1729@gmail.com

Email address: platypii@gmail.com